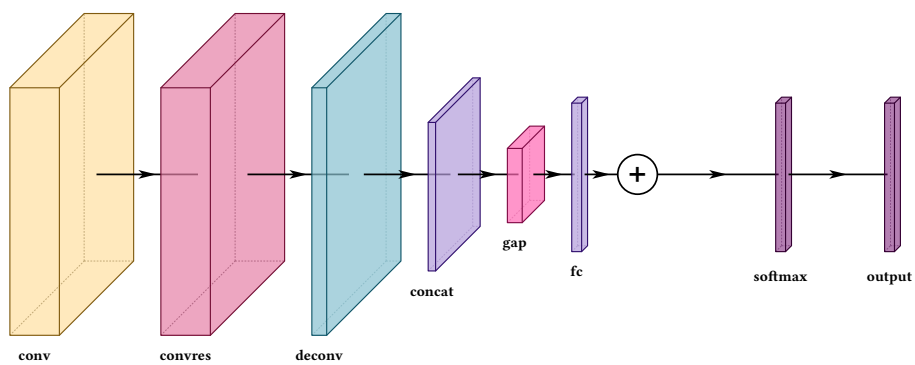


neural-netz

Neural network diagrams in Typst



Version 0.4 · J.C. Vaught · after Edgar Remy's neural-netz 0.3

Every figure in this manual is compiled from the code printed beside it.

Contents

1	What this is	3
1.1	How this manual is organised	3
1.2	Installing	3
2	Writing a layer	4
2.1	Options that do not exist are errors	4
2.2	Naming a layer	5
3	Layer types	6
3.1	Pooling attaches to the block in front	6
3.2	The sum node	7
3.3	The input layer	7
4	Sizing a block	8
4.1	Several bands in one block	8
4.2	Flat layers	8
4.3	Sizing from the tensor shape	8
5	Spacing	11
5.1	Spacing by hand	11
6	Labels	13
6.1	Channel numbers	13
6.2	Axis labels	13
6.3	Labelling the arrow between two blocks	13
6.4	Where a label sits	14
7	Images inside blocks	16
8	Colour	17
8.1	Activation bands	17
8.2	Palettes	18
9	Blocks of your own	19
9.1	Building a vocabulary	19
10	Connections	21
10.1	Routing modes	21
10.2	Lane heights	22
10.3	Where a route arrives	22
10.4	Telling routes apart	24
10.5	Labelling a route	25
10.6	Suppressing an axis arrow	25
11	Groups and repeats	26
11.1	Repeated blocks	27
12	Parallel branches	28
12.1	Stacking along the projection	28
12.2	Open at one end	29
12.3	Nesting	30
13	The figure as a whole	32
13.1	Legends	32
13.2	Fitting the page	32
13.3	Line weight and projection	32
14	Every option	34
14.1	Options by layer type	34
14.2	Connection options	34
14.3	Group options	35
14.4	Arguments to draw-network	35

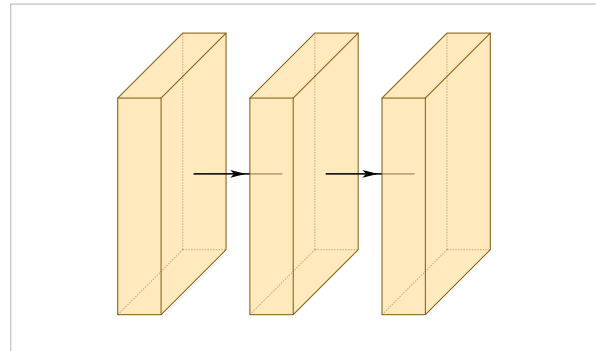
14.5 Coverage	35
15 Where to go next	36

What this is

neural-netz draws neural network architectures as isometric block diagrams, the kind that open a paper's method section. It is a Typst package built on [CeTZ](#), and its visual style comes from [PlotNeuralNet](#).

You describe the network as a list of layers. The package works out the geometry, the spacing, the arrows between blocks, the routing of any skip connection you add, and the legend.

```
#draw-network((
  conv(),
  conv(),
  conv(),
))
```



Three convolutions, no options. Sizes, spacing and arrows are all defaults.

1.1 How this manual is organised

Every feature is shown on its own, in the smallest figure that demonstrates it. There are no worked architectures here on purpose: complete models — AlexNet, VGG, ResNet, U-Net, FCN-8, YOLO26-n and six RGB-IR fusion variants — live in `examples/` in the repository, where you can read them as whole files rather than as fragments. This manual is for finding out what a single option does.

Chapters 2 to 5 cover a layer and its geometry. Chapters 6 to 9 cover everything written on or around a block. Chapters 10 to 13 cover the structures between blocks: connections, groups and parallel branches. Chapter 14 covers the figure as a whole, and chapter 15 lists every option in one place.

1.2 Installing

```
#import "@preview/neural-netz:0.4.0": draw-network, conv, pool, input
```

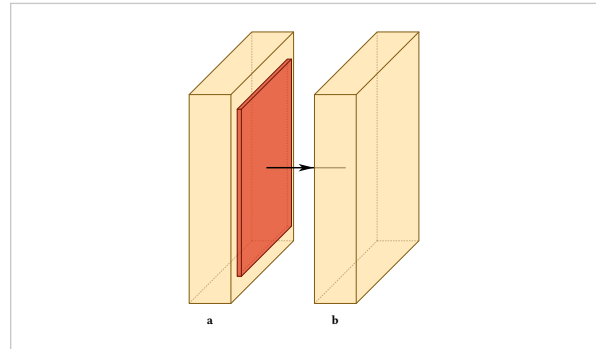
Every example in this manual assumes an import like that one. Import `draw-network` plus whichever layer constructors you use, or take the lot with `*`.

Version 0.4 is a fork and is not yet on Typst Universe, where the name `neural-netz` currently resolves to Edgar Remy's 0.3.0. Until it is published, point the import at a local copy: `#import "path/to/src/lib.typ": *`.

Writing a layer

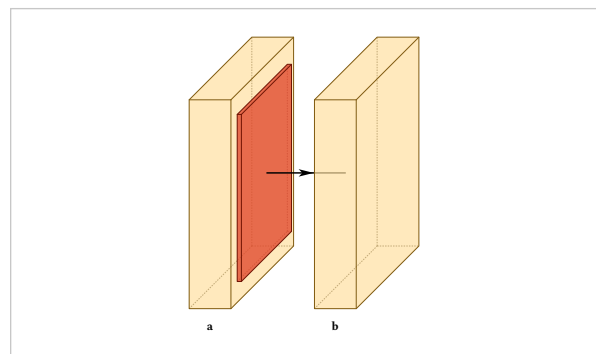
A network is an array of layers, passed to `draw-network`. A layer is a dictionary with a type and whatever options you want to set:

```
#draw-network((
  (type: "conv", label: "a"),
  (type: "pool",),),
  (type: "conv", label: "b"),
)
```



Every type also has a constructor, which is the same thing with the type filled in:

```
#draw-network((
  conv(label: "a"),
  pool(),
  conv(label: "b"),
)
```



Those two produce identical figures. Prefer the constructor form: its signature is exactly the set of options that type accepts, so an editor can list them while you type, and Typst rejects a misspelled argument by name before the package ever sees it. The dictionary form stays supported everywhere, including inside branches, and the two mix freely in one figure.

2.1 Options that do not exist are errors

An option a layer type does not read is rejected, not ignored:

```
error: unknown layer option "hieght" on layer 3 (type "conv").
Did you mean "height"? Options accepted here: bandfill, channels,
connection-label, depth, fill, height, image, label, ...
```

This matters more than it sounds. A key that is quietly ignored produces a figure that is merely wrong, and a wrong figure reads as the package being broken rather than as the typo it is. The check covers layer options, connection options, group options, unknown layer types, a layer with no type, and a connection or group naming a layer that has no such name.

The suggestion comes from an edit-distance search over the options that type actually accepts, so `widths` on a `pool` is caught as readily as `hieght` on a `conv` — the first is spelled correctly and still means nothing there.

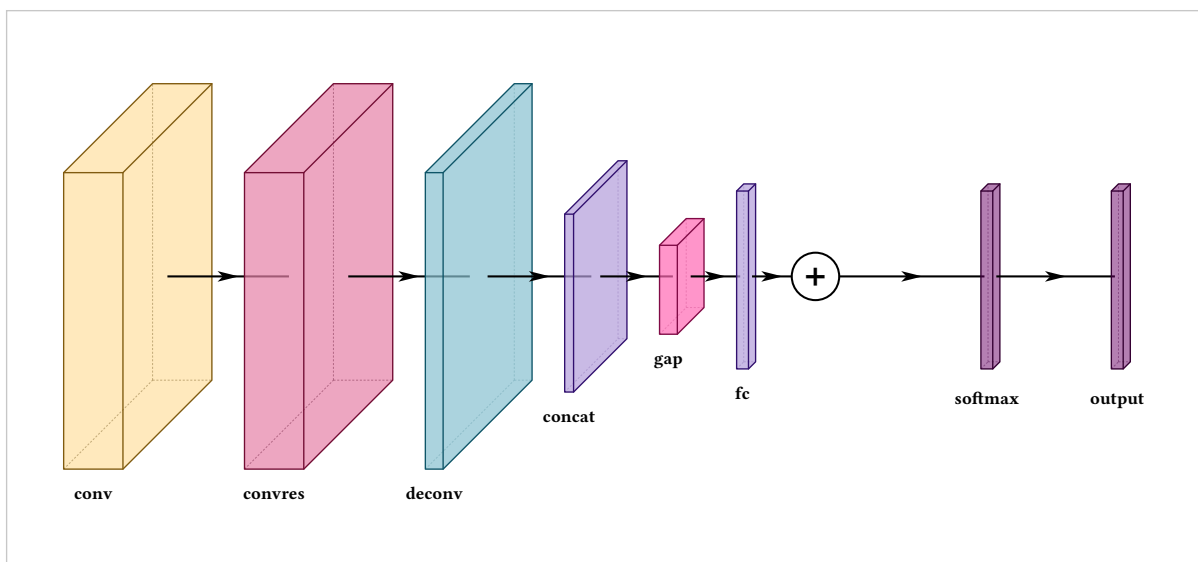
2.2 Naming a layer

Give a layer a name and other things can refer to it. Connections and groups both work by name, and nothing else does; a layer with no name is drawn and forgotten. Names are yours to choose and appear nowhere in the output.

Layer types

Fourteen types are predefined. Each carries a colour, a default size, a default legend entry, and in a few cases a behaviour.

```
#draw-network((
  conv(label: "conv"),
  convres(label: "convres"),
  deconv(label: "deconv"),
  concat(label: "concat"),
  gap(label: "gap"),
  fc(label: "fc"),
  sum(),
  softmax(label: "softmax"),
  output(label: "output", offset: 2),
))
```



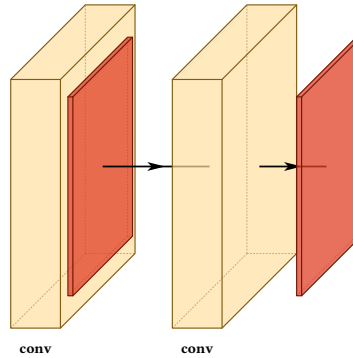
conv, convres, deconv, concat, gap, fc, sum, softmax, output. Also predefined: input, pool, unpool, convsoftmax, custom.

Type governs colour and defaults, not capability. Any type can be resized, recoloured, relabelled and given an image. If you want a block the package has no name for, use `custom` (chapter 9) rather than bending an unrelated type to fit.

3.1 Pooling attaches to the block in front

`pool` and `unpool` are the two types that are not free-standing blocks. They position themselves against the layer before them, which is what makes a convolution-plus-pool read as one stage. Give one an `offset` and it detaches:

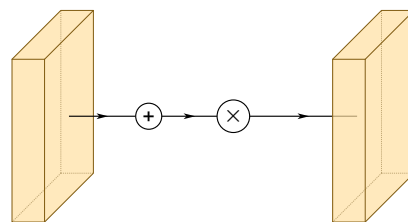
```
#draw-network((
  conv(label: "conv"),
  pool(), // attaches to the block before
  conv(label: "conv", offset: 2),
  pool(offset: 1.5), // an offset detaches it
))
```



3.2 The sum node

sum is a node rather than a block: it has a radius instead of a width, and a symbol you can replace.

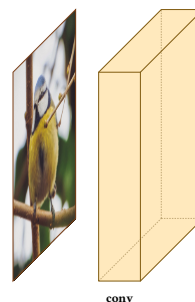
```
#draw-network((
  conv(),
  sum(),
  sum(symbol: [$times$], radius: 0.5),
  conv(),
))
```



3.3 The input layer

input defaults to a flat plane with no thickness, which is what an image is. Chapter 8 covers putting a picture on it.

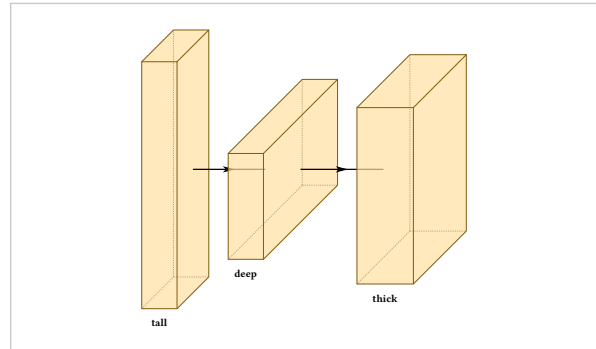
```
#draw-network((
  input(image: "default"),
  conv(label: "conv"),
))
```



Sizing a block

A block has three dimensions. height and depth are the spatial extent of the tensor; width (or widths) is its thickness, which by convention means channel count.

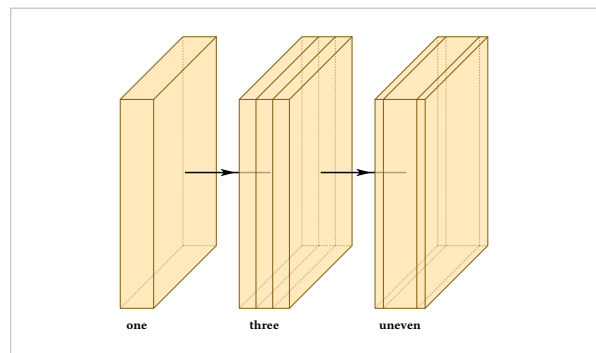
```
#draw-network((
  conv(label: "tall", height: 7, depth: 3),
  conv(label: "deep", height: 3, depth: 7),
  conv(label: "thick", widths: (1.6,)),
))
```



4.1 Several bands in one block

widths takes an array, and draws one band per entry inside a single block. That is how a stage of two or three convolutions at the same resolution is usually drawn: one block, banded, rather than three blocks in a row.

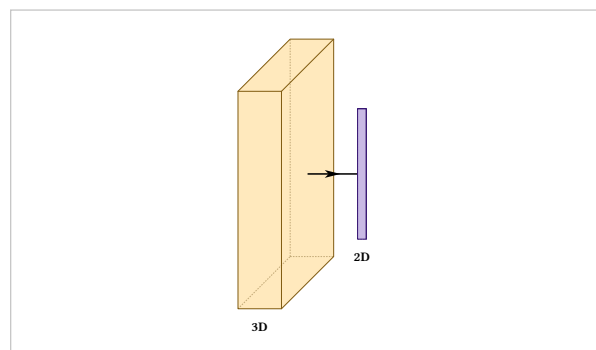
```
#draw-network((
  conv(label: "one", widths: (0.8,)),
  conv(label: "three", widths: (0.4, 0.4, 0.4)),
  conv(label: "uneven", widths: (0.2, 0.8, 0.2)),
))
```



4.2 Flat layers

depth: 0 draws a plain rectangle rather than a prism, which suits a fully-connected layer, where there is no spatial extent left to depict.

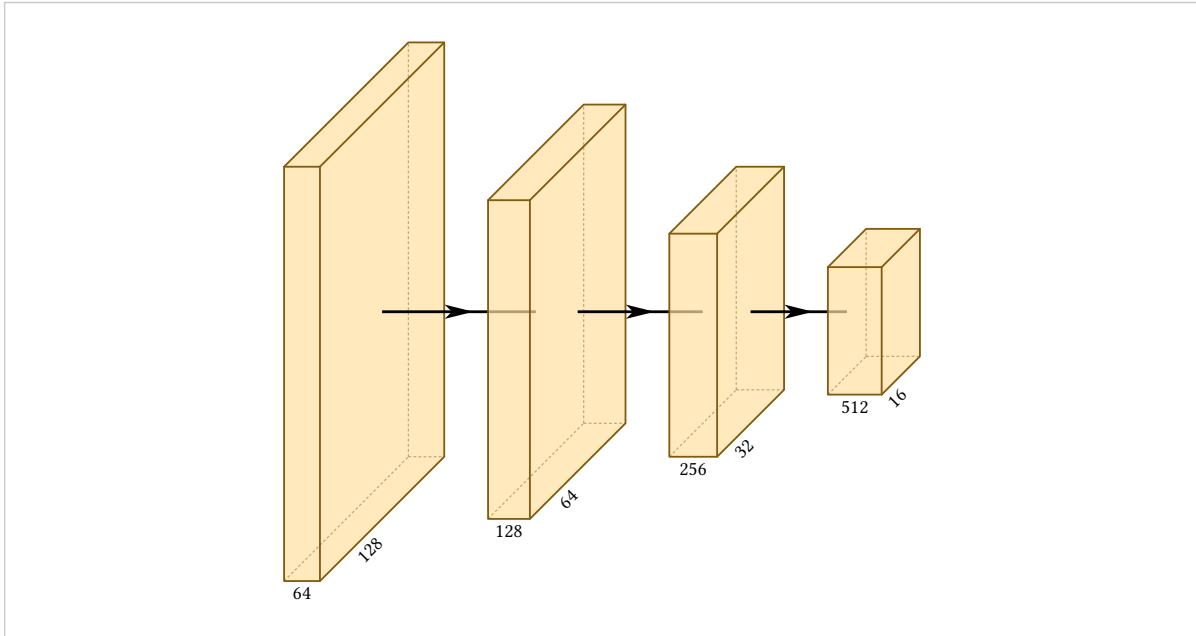
```
#draw-network((
  conv(label: "3D", depth: 4),
  fc(label: "2D", depth: 0),
))
```



4.3 Sizing from the tensor shape

Stating three numbers per block and keeping them consistent across a pyramid is the tedious part of drawing one of these. shape takes the tensor the layer produces, as (channels, height, width), and derives the geometry from it:

```
// (channels, height, width) of the tensor this layer produces
#draw-network((
  conv(shape: (64, 128, 128), channels: (64, 128)),
  conv(shape: (128, 64, 64), channels: (128, 64)),
  conv(shape: (256, 32, 32), channels: (256, 32)),
  conv(shape: (512, 16, 16), channels: (512, 16)),
))
```



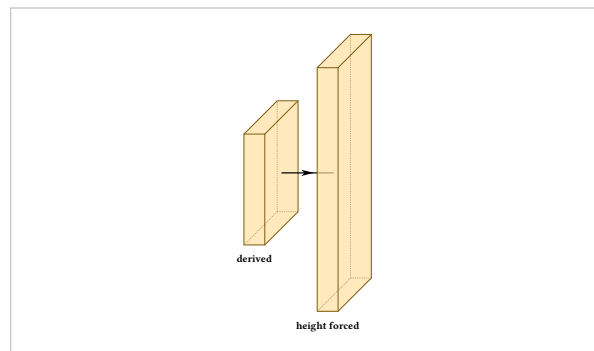
Both axes are logarithmic:

$$\text{height, depth} = 1.2 \log_2(\text{spatial}) - 3.2, \quad \text{width} = 0.075 \log_2(\text{channels})$$

Linear spatial extent does not work. Across a 640-to-20 pyramid it puts the smallest block at a quarter of a unit against 8 for the input, which is invisible.

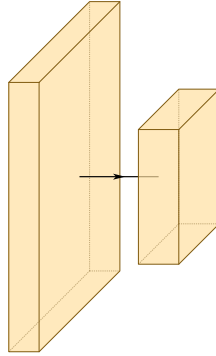
shape supplies defaults, not values. Anything you state explicitly wins, field by field, so a block can take its width and depth from its shape while its height is forced:

```
#draw-network((
  conv(label: "derived", shape: (256, 40, 40)),
  conv(
    label: "height forced",
    shape: (256, 40, 40),
    height: 7,
  ),
))
```



The constants are absolute rather than normalised across a figure, so the same shape gives the same size everywhere and two figures stay comparable. Change them with shape-scale on draw-network, which takes a slope and an intercept for each axis, applied to the base-2 logarithm:

```
#draw-network(  
  (conv(shape: (64, 128, 128)), conv(shape: (256, 32, 32))),  
  shape-scale: (  
    spatial: (2.0, -6.0),  
    channels: (0.15, 0.0),  
  ),  
)
```

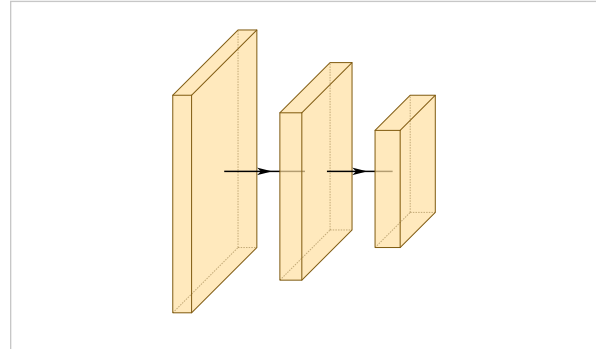


Only conv, convres and custom take a derived width, because only those three use thickness to mean channel count. The others have a fixed thickness that says something else, and keep it.

Spacing

offset is the gap between a layer and the one before it. It defaults to auto, which computes the gap from the drawing rather than from a constant:

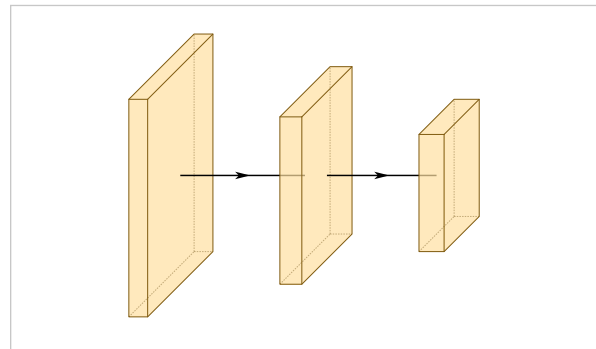
```
// No offsets stated
#draw-network((
  conv(shape: (64, 128, 128)),
  conv(shape: (128, 64, 64)),
  conv(shape: (256, 32, 32)),
))
```



Automatic spacing covers the previous block's isometric lean, adds a fixed strip of white space, and widens where a connection descends into the gap. A layer drawn in isometric projection leans right by $\text{depth} \times \text{depth-multiplier}$, so a gap narrower than that lean buys no visible space at all — which is why a constant offset works for one size of block and fails for the next.

Set the strip of white space with auto-gap on draw-network:

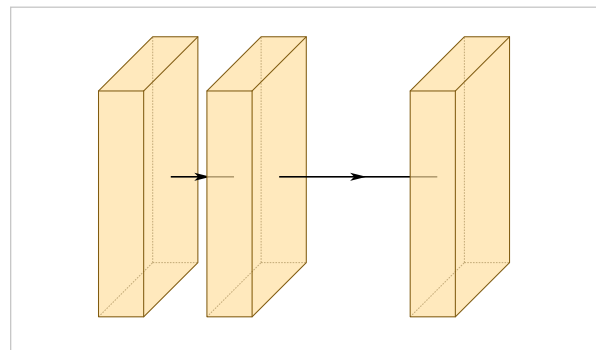
```
#draw-network(
  (
    conv(shape: (64, 128, 128)),
    conv(shape: (128, 64, 64)),
    conv(shape: (256, 32, 32)),
  ),
  auto-gap: 1.6,
)
```



5.1 Spacing by hand

A number overrides the automatic gap for one layer, and means what it always meant:

```
#draw-network((
  conv(depth: 4),
  conv(depth: 4, offset: 1.4),
  conv(depth: 4, offset: 3.5),
))
```



Two helpers are exported for figures that compute their own geometry:

```
depth-shear(6)    // 1.8 -- how far a depth-6 block leans right
min-clear-offset(6) // 3.6 -- smallest gap a connection can descend into
```

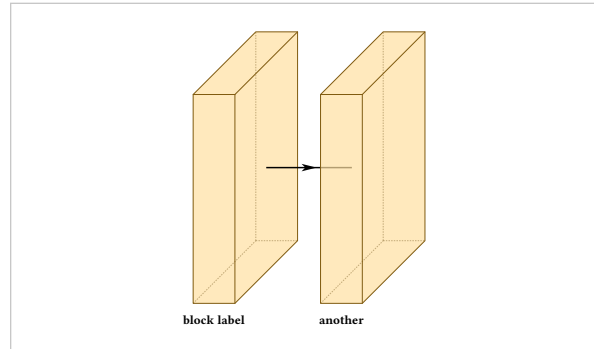
A connection descending between two blocks arrives at the midpoint of the arrow joining them, so it clears the lean only when half the offset exceeds it. That is what `min-clear-offset` returns, and what automatic spacing applies for you.

Both take a `depth-multiplier` argument, which must match the one passed to `draw-network`. The default multiplier of 0.3 has no exact binary representation, so `depth-shear(4.5)` is 1.3499999999999999. Compare with a tolerance, if you compare at all.

Labels

label names a block, beneath it.

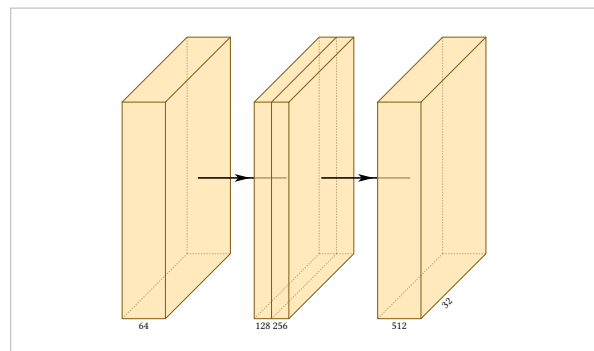
```
#draw-network((
  conv(label: "block label"),
  conv(label: "another"),
))
```



6.1 Channel numbers

channels prints numbers along the top of a block, one per band. One extra entry beyond the number of bands becomes the diagonal axis label instead, which is where the spatial resolution usually goes.

```
#draw-network((
  // one number per band
  conv(channels: (64,)),
  conv(channels: (128, 256), widths: (0.4, 0.4)),
  // one extra: the diagonal axis label
  conv(channels: (512, 32)),
))
```

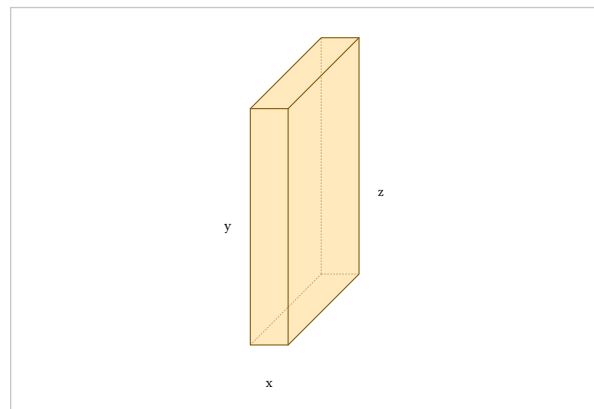


Strings work as well as numbers, so channels: ("", "1/16") labels the axis and nothing else.

6.2 Axis labels

xlabel, ylabel and zlabel name the three axes of one block, for the figure that has to explain the projection itself.

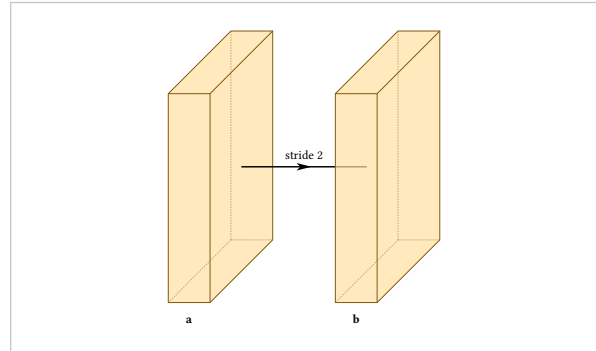
```
#draw-network((
  conv(
    xlabel: "x", ylabel: "y", zlabel: "z",
    widths: (0.8,),
  ),
))
```



6.3 Labelling the arrow between two blocks

connection-label sits on the axis arrow **after** the layer that declares it.

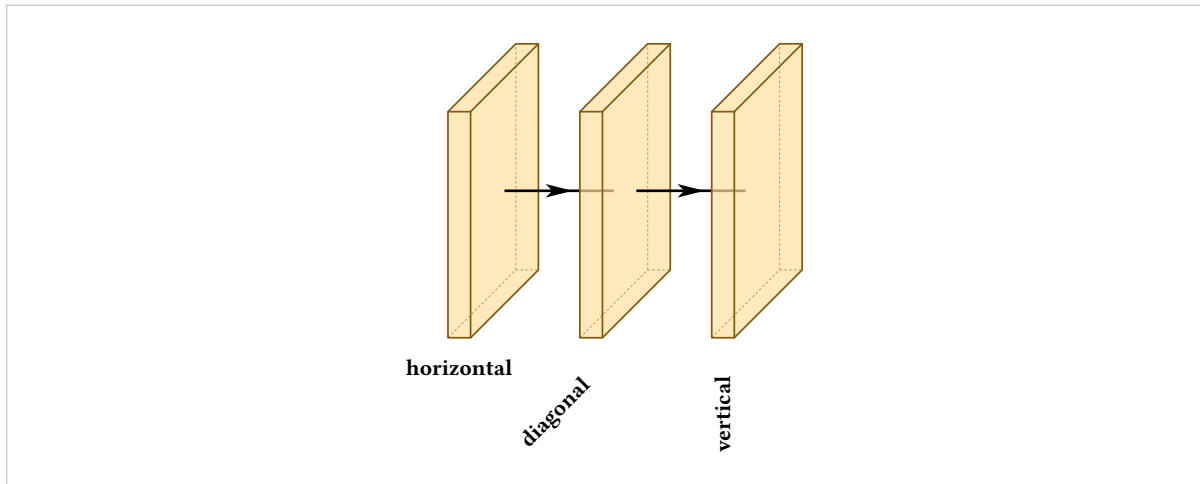
```
#draw-network((
  conv(label: "a", connection-label: "stride 2"),
  conv(label: "b", offset: 3),
))
```



6.4 Where a label sits

Labels sit at a fixed spot beneath each block, so long labels on closely spaced blocks collide. Rather than spreading the blocks apart to solve a typesetting problem, move the label. `label-orient` has three presets:

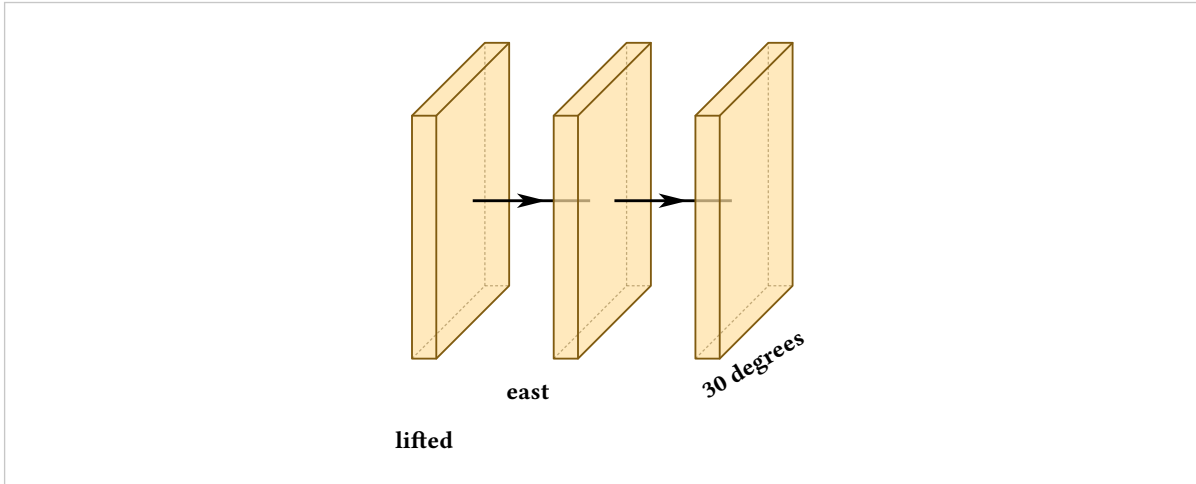
```
#draw-network((
  conv(label: "horizontal", height: 3, depth: 3, widths: (0.3,)),
  conv(label: "diagonal", height: 3, depth: 3, widths: (0.3,)), label-orient: "diagonal"),
  conv(label: "vertical", height: 3, depth: 3, widths: (0.3,)), label-orient: "vertical"),
))
```



Each preset pairs its angle with the anchor that places it correctly, so a diagonal label points its end at its own block and a vertical one hangs centred beneath it, with no manual adjustment. Rotating trades horizontal space, which is scarce, for vertical space, which is usually free.

For anything else there are `label-dx`, `label-dy`, `label-anchor` and `label-angle`:

```
#draw-network((
  conv(label: "lifted", height: 3, depth: 3, widths: (0.3,)), label-dy: -0.6),
  conv(label: "east", height: 3, depth: 3, widths: (0.3,)),
    label-anchor: "base-east", label-dx: -0.2),
  conv(label: "30 degrees", height: 3, depth: 3, widths: (0.3,)),
    label-angle: 30deg, label-anchor: "base-west"),
))
```



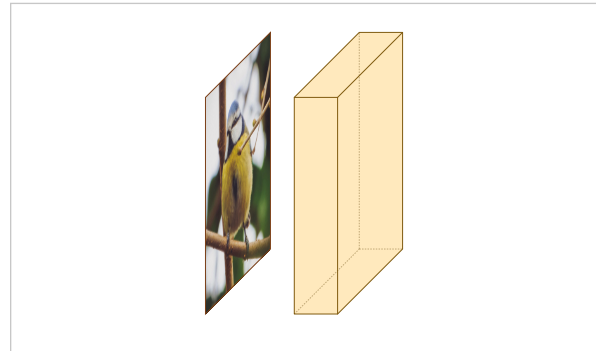
Labels are anchored on their baseline rather than on their bounding box, so a label with descenders sits level with one without. Use "base-east" and "base-west" to anchor horizontally while keeping that alignment.

Setting both label-orient and label-angle is an error, and label-orient accepts only "horizontal", "diagonal" and "vertical". For any other angle use label-angle and pick label-anchor yourself, since the right anchor for an arbitrary rotation depends on where you want the text to sit.

Images inside blocks

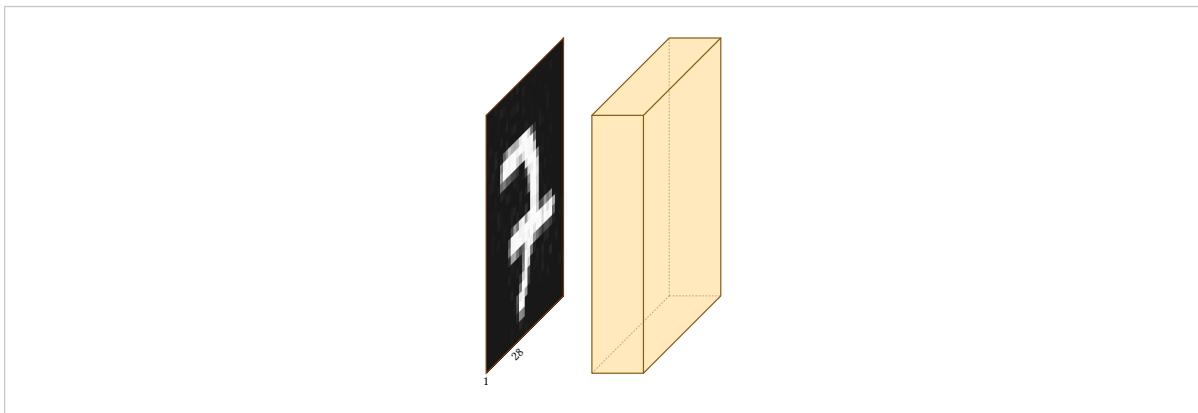
image puts content on a block's face, sheared to match the projection. The string "default" uses the bundled photograph:

```
#draw-network((
  input(image: "default"),
  conv(),
))
```



Any Typst image works:

```
#let mnist = "../examples/networks/mnist-img-sample.jpg"
#draw-network((
  input(image: image(mnist), channels: (1, 28)),
  conv(),
))
```



And it does not have to be an image. Any content is accepted, which is useful for putting an operator symbol on a block:

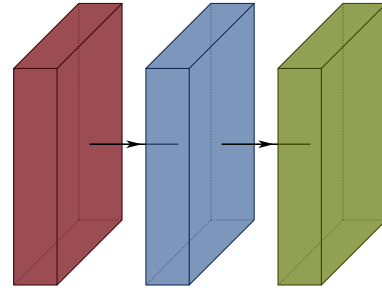
```
// Any content works, not only an image
#draw-network((
  custom(image: [#text(20pt)[$Sigma$]]),
  custom(image: [hello]),
))
```



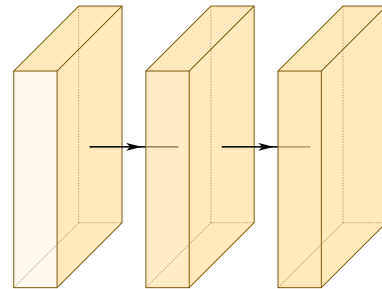
Colour

`fill` sets a block's colour and `opacity` how solid it is. Edge colours are derived from the fill, so a recoloured block stays internally consistent.

```
#draw-network((
  conv(fill: rgb("#73000A")),
  conv(fill: rgb("#466A9F")),
  conv(fill: rgb("#65780B")),
))
```



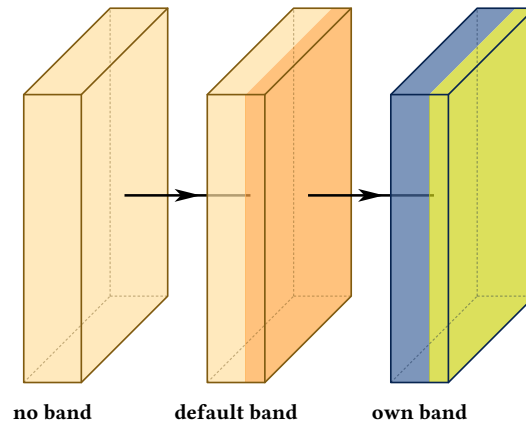
```
#draw-network((
  conv(opacity: 0.2),
  conv(opacity: 0.6),
  conv(opacity: 1.0),
))
```



8.1 Activation bands

A darker band at the end of a convolution conventionally marks the activation. Turn them on for a figure with `show-relu: true` on `draw-network`, and override per layer:

```
#draw-network(
  (
    conv(label: "no band", height: 4, depth: 4, widths: (0.8,)), show-relu: false),
    conv(label: "default band", height: 4, depth: 4, widths: (0.8,)),
    conv(label: "own band", height: 4, depth: 4, widths: (0.8,),
      fill: rgb("#466A9F"), bandfill: rgb("#CED318")),
  ),
  show-relu: true,
)
```

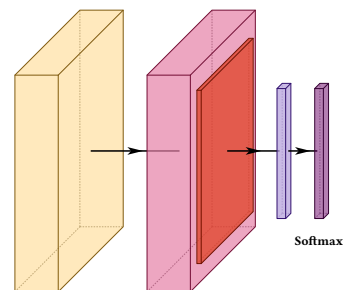


An undeclared band colour is derived from the layer's own fill, so a recoloured block gets a band that matches it rather than one from the palette.

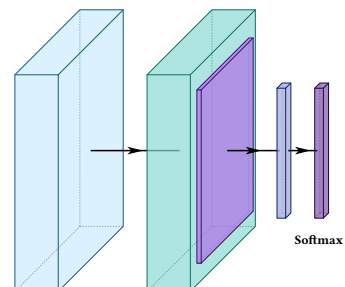
8.2 Palettes

Two palettes ship with the package, "warm" (the default) and "cold".

```
#draw-network(
  (conv(), convres(), pool(), fc(), softmax()),
  palette: "warm",
)
```



```
#draw-network(
  (conv(), convres(), pool(), fc(), softmax()),
  palette: "cold",
)
```

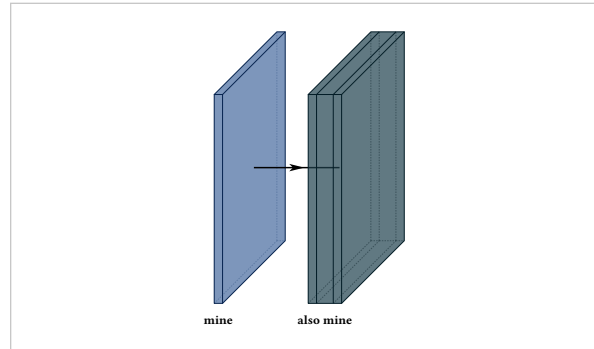


A palette sets the defaults only. Any fill you state wins, so a figure can sit on a palette and still recolour three blocks to your own scheme.

Blocks of your own

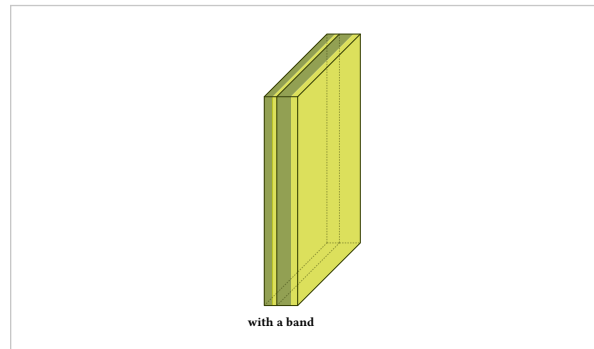
custom is the generic block: no meaning attached, every option available.

```
#draw-network((
  custom(label: "mine", fill: rgb("#466A9F")),
  custom(
    label: "also mine",
    widths: (0.2, 0.4, 0.2),
    fill: rgb("#1F414D"),
  ),
))
```



It takes bands and an activation band like a convolution does:

```
#draw-network((
  custom(
    label: "with a band",
    widths: (0.3, 0.5),
    fill: rgb("#65780B"),
    bandfill: rgb("#CED318"),
    show-relu: true,
  ),
))
```

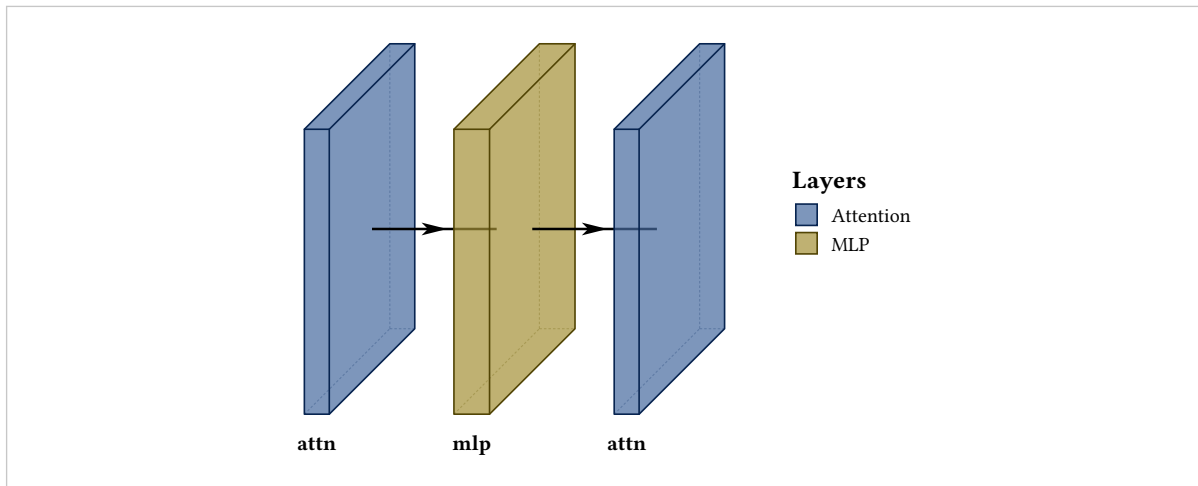


9.1 Building a vocabulary

A custom layer with its options fixed is just a function returning a dictionary, so your own block types are a few lines. This is the intended way to draw something the package has no name for — attention, a gate, a state-space block — rather than repurposing a type that means something else.

```
// Your own block type is a function returning a layer
#let attention(..args) = custom(
  fill: rgb("#466A9F"), width: 0.35, height: 4, depth: 4,
  legend: "Attention", ..args,
)
#let mlp(..args) = custom(
  fill: rgb("#A49137"), width: 0.5, height: 4, depth: 4,
  legend: "MLP", ..args,
)

#draw-network(
  (attention(label: "attn"), mlp(label: "mlp"), attention(label: "attn")),
  show-legend: true,
)
```

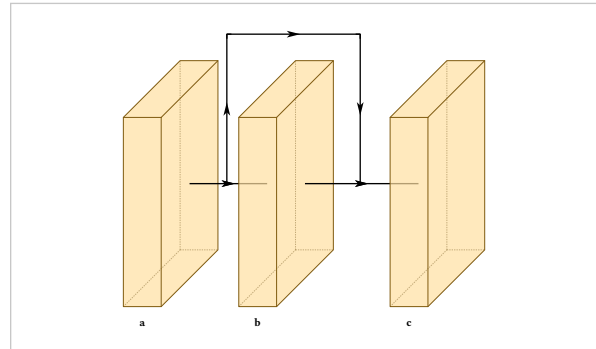


Give the constructor a legend and every block made from it shares one legend entry.

Connections

Arrows along the main axis are drawn for you. Anything else — a skip, a residual add, a lateral feed, an auxiliary path — goes in connections, as a list of routes between two named layers.

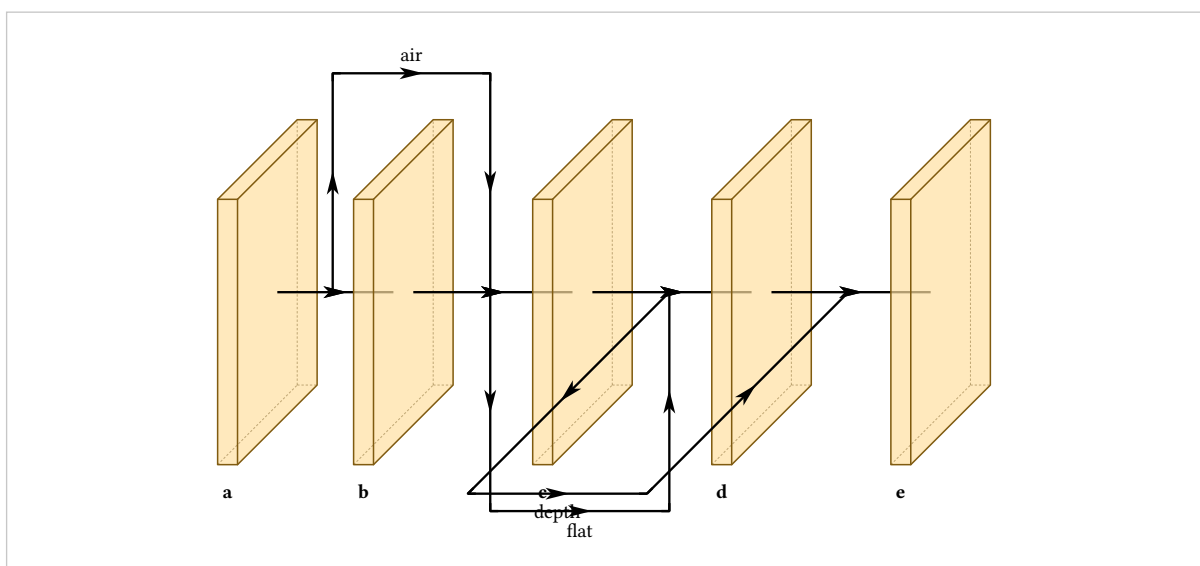
```
#draw-network(
(
  conv(name: "a", label: "a"),
  conv(name: "b", label: "b"),
  conv(name: "c", label: "c"),
),
connections: ((from: "a", to: "c")),
)
```



10.1 Routing modes

mode picks where a route runs. "air" goes over the top and is the default, "flat" underneath, "depth" along the projection's own axis.

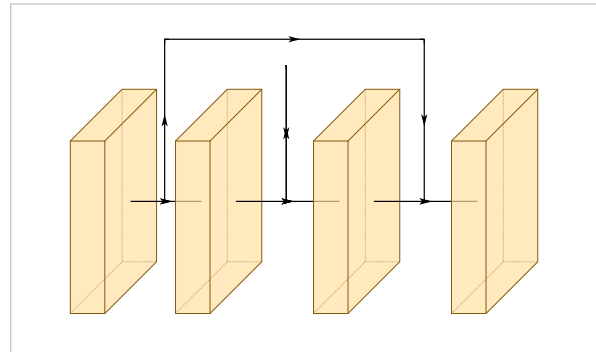
```
#draw-network(
(
  conv(name: "a", label: "a", height: 4, depth: 4, widths: (0.3,)),
  conv(name: "b", label: "b", height: 4, depth: 4, widths: (0.3,)),
  conv(name: "c", label: "c", height: 4, depth: 4, widths: (0.3,)),
  conv(name: "d", label: "d", height: 4, depth: 4, widths: (0.3,)),
  conv(name: "e", label: "e", height: 4, depth: 4, widths: (0.3,)),
),
connections: (
  (from: "a", to: "c", mode: "air", label: "air"),
  (from: "b", to: "d", mode: "flat", label: "flat"),
  (from: "c", to: "e", mode: "depth", label: "depth"),
),
)
```



10.2 Lane heights

`pos` is how far the route sits from the centre axis. It defaults to `auto`, which ranks routes by how far they reach: a route spanning more blocks sits higher, so a longer route always arcs over a shorter one instead of crossing it.

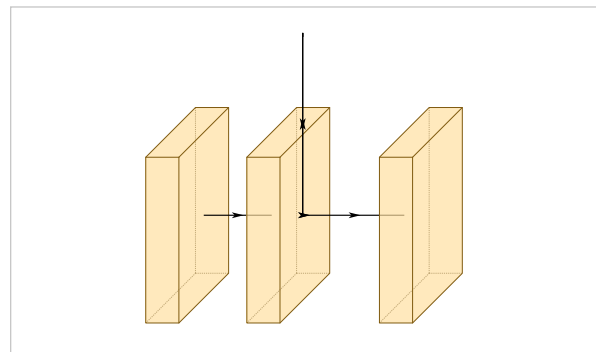
```
#draw-network(  
  (  
    conv(name: "a"), conv(name: "b"),  
    conv(name: "c"), conv(name: "d"),  
  ),  
  connections: (  
    // the longer route arcs over the shorter  
    (from: "a", to: "d"),  
    (from: "b", to: "c"),  
  ),  
)
```



Routes of equal reach share a height. Where two of them overlap, the second goes to the opposite side of the axis at the same height rather than being pushed further out than its reach warrants. `lane-unit` on `draw-network` sets the spacing between heights, and `clearance` on a route sets how far the lowest one sits from the blocks.

A number overrides all of that for one route, measured from the axis:

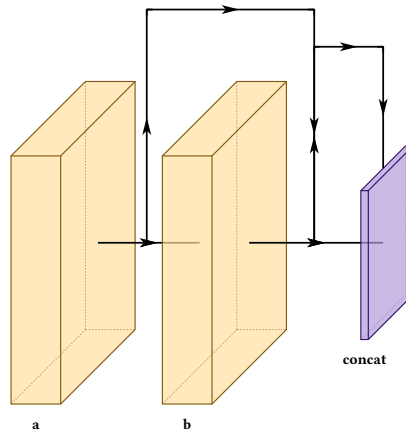
```
#draw-network(  
  (conv(name: "a"), conv(), conv(name: "c")),  
  connections: ((from: "a", to: "c", pos: 5.5),),  
)
```



10.3 Where a route arrives

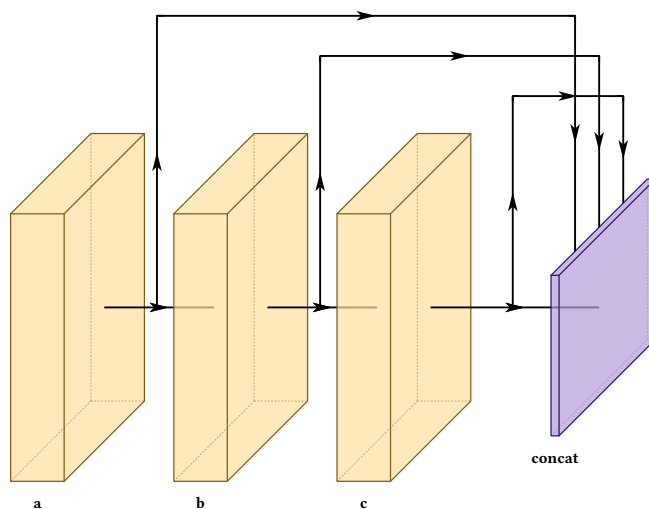
By default a route arrives on the main axis just before its target. `touch-layer` lands it on the target block itself, choosing a side from the routing mode: `air` arrives on the top edge, `flat` on the bottom, `depth` on the left.

```
#draw-network(
(
  conv(name: "a", label: "a"), conv(name: "b", label: "b"),
  concat(name: "cat", label: "concat"),
),
connections: (
  (from: "a", to: "cat"),           // arrives on the axis arrow
  (from: "b", to: "cat", touch-layer: true), // arrives on the block itself
),
)
```



Several routes landing on one block would stack their arrowheads on the same point, so arrive-offset spreads them along whichever edge the mode chose. It defaults to auto, which spaces a whole fan evenly and leaves a lone arrival centred:

```
#draw-network(
(
  conv(name: "a", label: "a"), conv(name: "b", label: "b"),
  conv(name: "c", label: "c"), concat(name: "cat", label: "concat", depth: 6),
),
connections: (
  (from: "a", to: "cat", touch-layer: true),
  (from: "b", to: "cat", touch-layer: true),
  (from: "c", to: "cat", touch-layer: true),
),
)
```



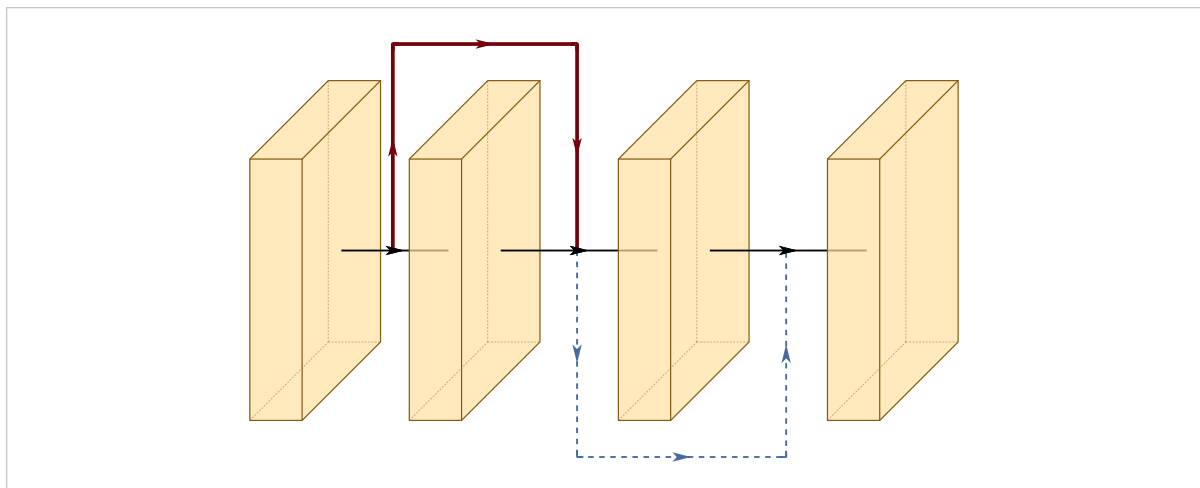
Routes are grouped by the edge they land on, then spread across it: k routes divide the edge into $k + 1$ intervals and sit at the interior boundaries, so the outermost pair is inset rather than sitting on the corners. They are ordered by where each route starts, so a fan does not cross itself, and adding a route respaces the rest.

A route arriving on the bottom or left edge has its final stretch drawn behind the block, because those edges are on the far side and the route genuinely passes underneath before reaching them. Blocks are semi-transparent, so it shows faintly rather than disappearing.

10.4 Telling routes apart

A residual add, a concat feed and an auxiliary supervision path are different things, and by default they all look the same. color, dash and thickness separate them:

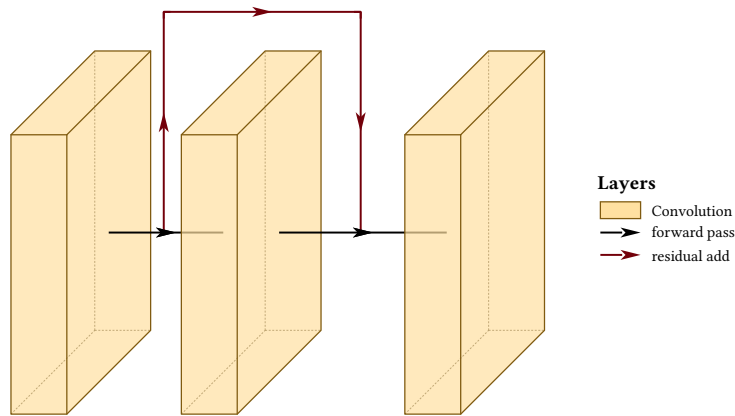
```
#draw-network(
  (conv(name: "a"), conv(name: "b"), conv(name: "c"), conv(name: "d")),
  connections: (
    (from: "a", to: "c", color: rgb("#73000A"), thickness: 2),
    (from: "b", to: "d", color: rgb("#466A9F"), dash: "dashed"),
  ),
)
```



thickness multiplies rather than replaces, so a figure that sets stroke-thickness on draw-network still scales its routes. Arrowheads take the line colour, since a coloured line with black arrowheads reads as a bug rather than a choice.

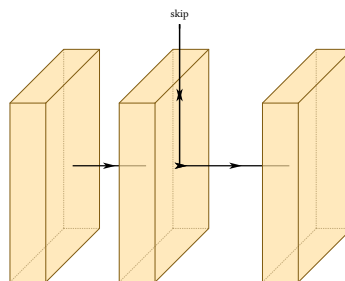
Give a route a legend and it joins the legend as a line sample rather than a colour swatch, since what distinguishes a route is its stroke and not a fill. main-legend names the automatic axis arrows, which otherwise go unexplained:

```
#draw-network(
  (conv(name: "a"), conv(name: "b"), conv(name: "c")),
  connections: (
    (from: "a", to: "c", color: rgb("#73000A"), legend: "residual add"),
  ),
  show-legend: true,
  main-legend: "forward pass",
)
```



10.5 Labelling a route

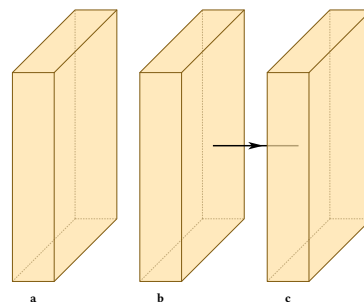
```
#draw-network(
  (conv(name: "a"), conv(), conv(name: "c")),
  connections: ((from: "a", to: "c", label: "skip")),
)
```



10.6 Suppressing an axis arrow

`show-connection: false` removes the arrow **after** a layer. The input layer defaults to false, which is why an image usually sits detached; set it true to join it up.

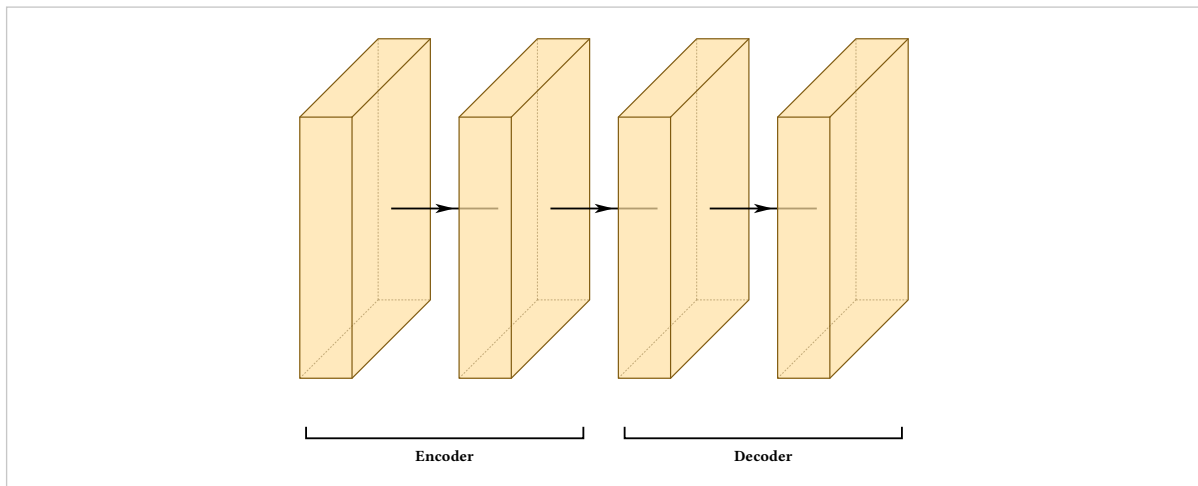
```
#draw-network((
  // no arrow after this one
  conv(label: "a", show-connection: false),
  conv(label: "b"),
  conv(label: "c"),
))
```



Groups and repeats

Backbone, neck and head are the phrases anyone uses out loud to explain one of these figures. groups draws them, as labelled brackets under a span of layers.

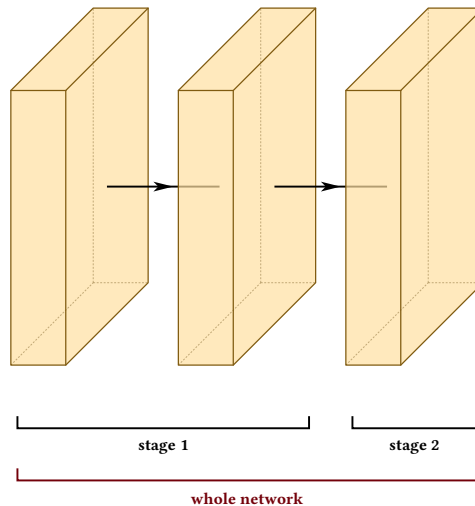
```
#draw-network(
  (
    conv(name: "e1"), conv(name: "e2"),
    conv(name: "d1"), conv(name: "d2"),
  ),
  groups: (
    (from: "e1", to: "e2", label: "Encoder"),
    (from: "d1", to: "d2", label: "Decoder"),
  ),
)
```



Each entry spans from one named layer to another, inclusive, and the two may be the same layer. The bracket covers the drawn footprint rather than the front faces, so it sits under the whole block including its lean, and its ends are inset slightly so two adjacent groups read as two rather than as one continuous rule.

Brackets are placed below everything else in the figure, including a route running underneath the stack. `offset` pushes one further down, which is how you stack a group enclosing other groups onto its own row, and `color` tints one:

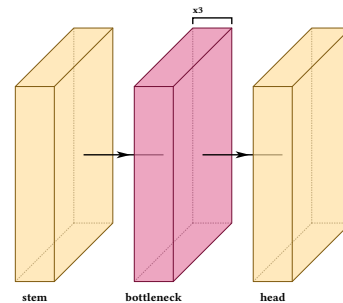
```
#draw-network(
  (conv(name: "a"), conv(name: "b"), conv(name: "c")),
  groups: (
    (from: "a", to: "b", label: "stage 1"),
    (from: "c", to: "c", label: "stage 2"),
    (from: "a", to: "c", label: "whole network", offset: 2.1,
     color: rgb("#73000A")),
  ),
)
```



11.1 Repeated blocks

Depth-scaled models stack the same block several times. Writing that as extra entries in widths makes it **look** repeated, but the package has no idea it is: it cannot label the repeat or bracket it, and the figure claims one wide block rather than several. repeat declares it:

```
#draw-network((
  conv(label: "stem"),
  convs(label: "bottleneck", repeat: 3),
  conv(label: "head"),
))
```



The block is drawn once, with a bracket above it carrying the count.

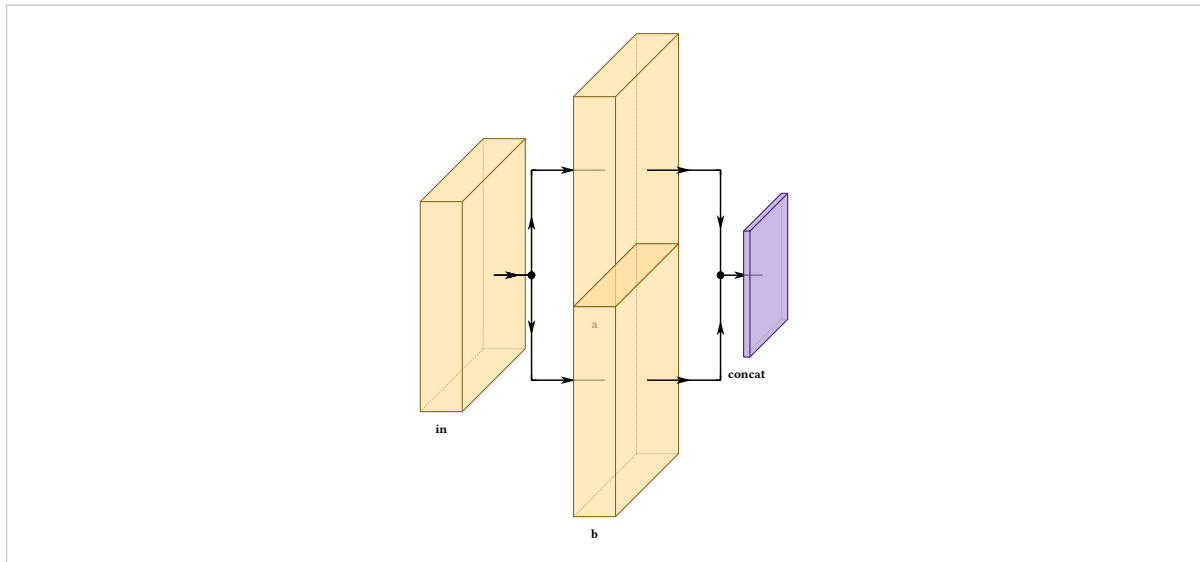
Ghosted copies were tried first and rejected. An outline behind the block reads as an empty box rather than as another one of the same block, and drawing N of them is either misleading about the count or unreadable once N is large. A bracket states the count instead of depicting it, costs no horizontal space, and stays legible at any N.

repeat is ignored on pool, unpool and sum: the first two attach to the block before them rather than being blocks in their own right, and a sum is a node, not a stack. It also describes one layer entry and not a run of them, so a repeated pair such as attention-then-MLP cannot be expressed with it.

Parallel branches

`draw-network` advances one cursor along one axis, so anything genuinely parallel — two detection heads, the two paths inside a CSP block, a two-stream fusion — would have to be collapsed into a single block with arrows pointed at it. A branch entry draws it as it is. Each branch is a layer list of its own.

```
#draw-network((
  conv(label: "in"),
  branch(spread: 5, branches: (
    (conv(label: "a"),),
    (conv(label: "b"),),
  )),
  concat(label: "concat"),
))
```

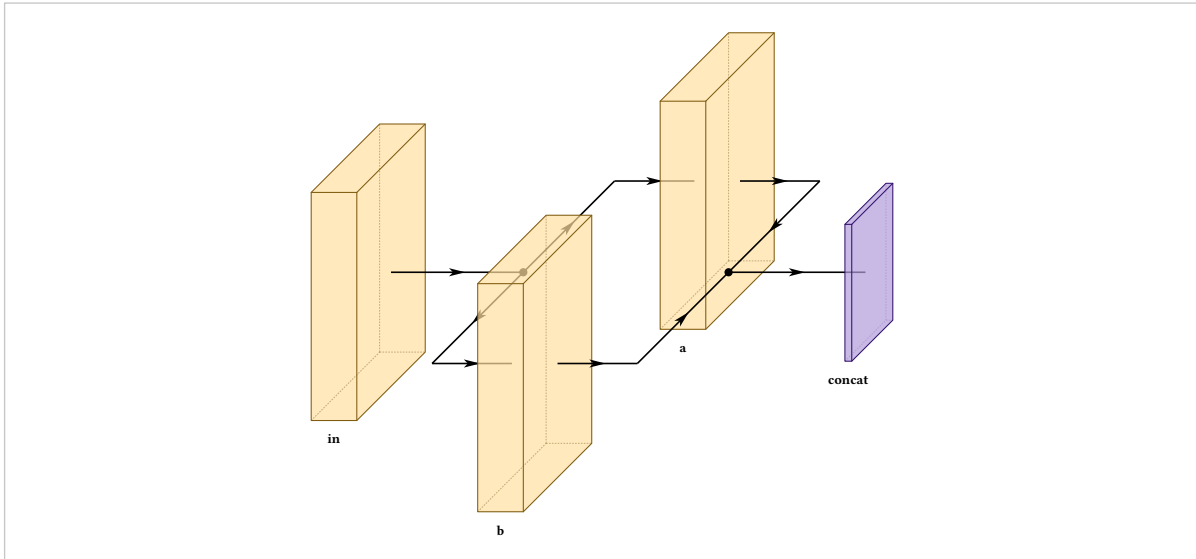


`spread` sets the separation, centred on the trunk, and `lead` how far the fan-out and rejoin arrows run. An odd count puts one branch on the trunk line; an even count leaves it empty. A filled dot marks each point where flow divides or meets.

12.1 Stacking along the projection

`spread-mode: "depth"` stacks the branches along the projection's 45-degree axis instead of straight up, so they read as parallel copies sitting behind one another. The fan-out and rejoin become two parallel spines with horizontal teeth — a parallelogram rather than the mirrored hexagon a vertical split produces. Neither 45 is a free angle: it is the one the projection already uses.

```
#draw-network((
  conv(label: "in"),
  branch(spread: 4, spread-mode: "depth", branches: (
    (conv(label: "a")),
    (conv(label: "b")),
  )),
  concat(label: "concat"),
))
```

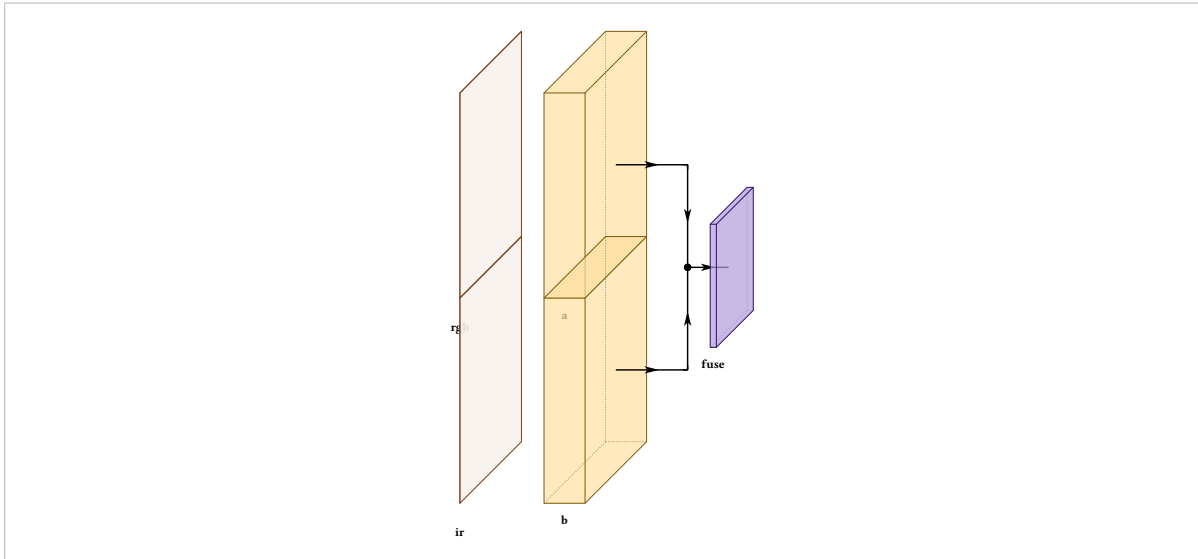


rejoin-lead widens the rejoin side alone, which depth mode often wants, since its return spine descends past each block's lower-right corner — which is exactly where diagonal dimension labels sit.

12.2 Open at one end

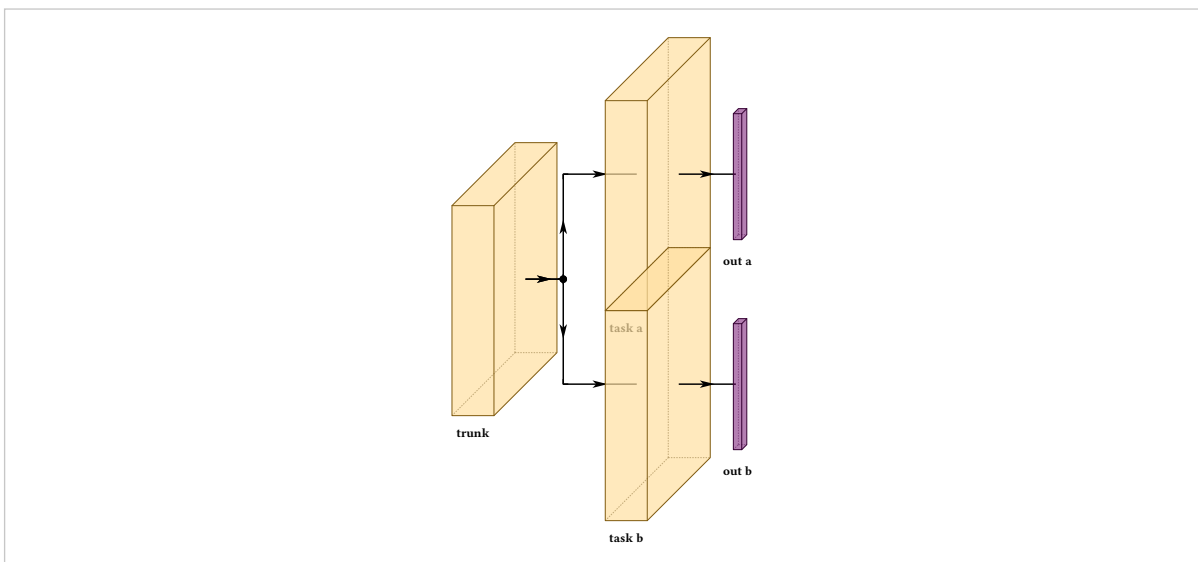
A branch may be open at one end. open: "start" draws no fan-out, so the branches simply begin. A branch with nothing before it is open at the start by definition, which is how a multi-input network starts:

```
// Nothing before it, so the branches simply begin: a two-input network
#draw-network((
  branch(spread: 5, branches: (
    (input(label: "rgb"), conv(label: "a")),
    (input(label: "ir"), conv(label: "b")),
  )),
  concat(label: "fuse"),
))
```



open: "end" draws no rejoin, so one trunk fans out into independent outputs:

```
#draw-network((
  conv(label: "trunk"),
  branch(spread: 5, open: "end", branches: (
    (conv(label: "task a"), output(label: "out a")),
    (conv(label: "task b"), output(label: "out b")),
  )),
))
```



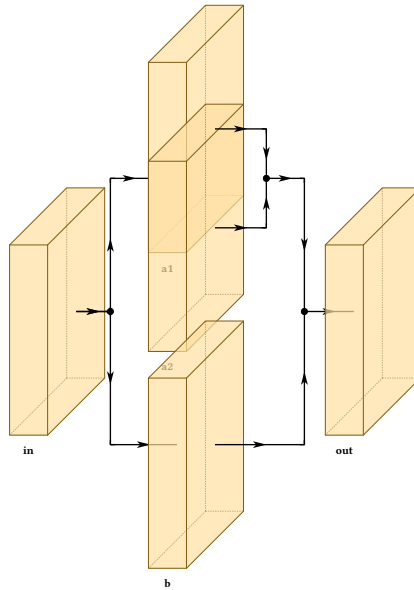
12.3 Nesting

A branch may contain branches, since walking a branch is the same operation as walking the trunk.

```

#draw-network((
  conv(label: "in"),
  branch(spread: 7, branches: (
    (
      branch(spread: 2.6, lead: 1.2, branches: (
        (conv(label: "a1"),),
        (conv(label: "a2"),),
      )),
    (conv(label: "b"),),
  )),
  conv(label: "out"),
))

```



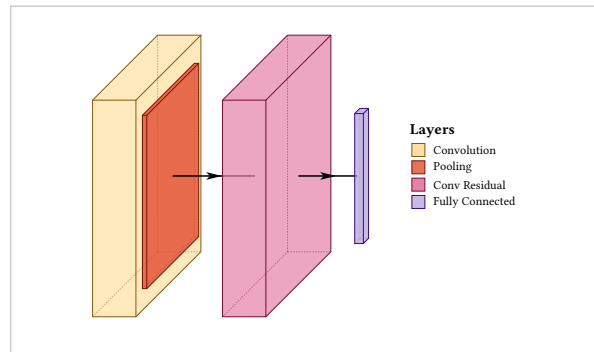
The rejoin waits for the longest branch rather than cutting the others short. Named layers inside a branch join the same table the trunk uses, so connections, groups and automatic lane ranking all reach into branches, and a group naming any layer inside a branch widens to the branch's whole drawn extent, plumbing included.

The figure as a whole

13.1 Legends

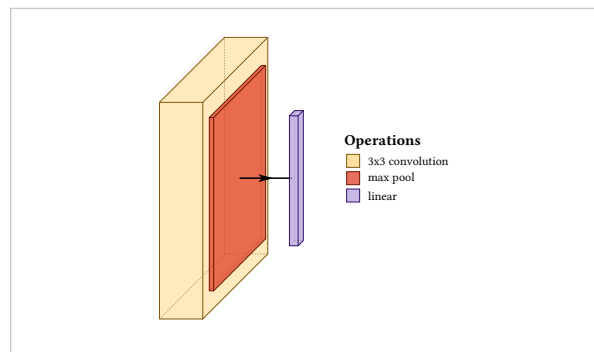
`show-legend: true` collects every layer type used into a legend, in order of first appearance.

```
#draw-network(
  (conv(), pool(), convres(), fc()),
  show-legend: true,
)
```



`legend` on a layer overrides the default name for its type, and `legend-title` renames the whole box:

```
#draw-network(
  (
    conv(legend: "3x3 convolution"),
    pool(legend: "max pool"),
    fc(legend: "linear"),
  ),
  show-legend: true,
  legend-title: "Operations",
)
```

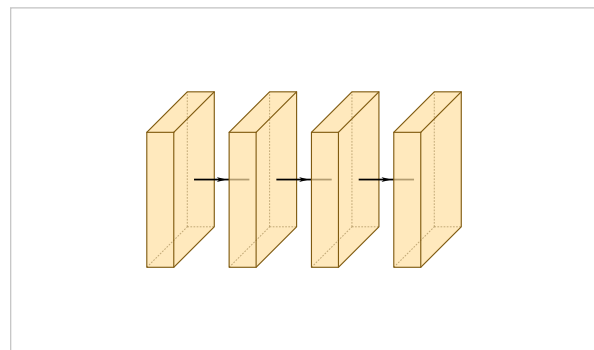


Layers sharing a legend name appear once. Connection legend entries (chapter 10) appear as line samples, and the sample column widens when any is present so a dash pattern has room to read.

13.2 Fitting the page

A figure is drawn at its natural size, which for a long network is wider than a page. `scale` shrinks the whole thing, text included:

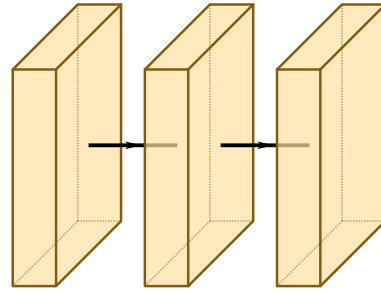
```
#draw-network(
  (conv(), conv(), conv(), conv()),
  scale: 55%,
)
```



13.3 Line weight and projection

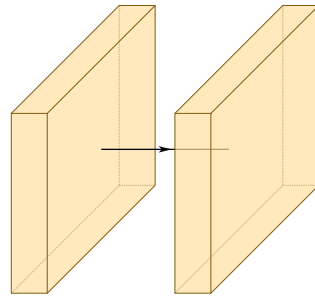
`stroke-thickness` multiplies every stroke in the figure, which matters when a figure is reduced for a two-column layout and hairlines start dropping out:

```
#draw-network(  
  (conv(), conv(), conv()),  
  stroke-thickness: 2.5,  
)
```



depth-multiplier controls how far blocks lean, which is the strength of the isometric projection:

```
#draw-network(  
  (conv(), conv()),  
  depth-multiplier: 0.6,  
)
```



Every option

The tables below are generated from the package itself rather than written out here, so an option cannot exist without appearing in this chapter.

14.1 Options by layer type

Type	Options
branch	branches, lead, name, open, rejoin-lead, spread, spread-mode
concat	channels, connection-label, depth, fill, height, image, label, label-anchor, label-angle, label-dx, label-dy, label-orient, legend, name, offset, opacity, repeat, shape, show-connection, width
conv	bandfill, channels, connection-label, depth, fill, height, image, label, label-anchor, label-angle, label-dx, label-dy, label-orient, legend, name, offset, opacity, repeat, shape, show-connection, show-relu, widths, xlabel, ylabel, zlabel
convres	bandfill, channels, connection-label, depth, fill, height, image, label, label-anchor, label-angle, label-dx, label-dy, label-orient, legend, name, offset, opacity, repeat, shape, show-connection, show-relu, widths, xlabel, ylabel, zlabel
convsoftmax	channels, connection-label, depth, fill, height, image, label, label-anchor, label-angle, label-dx, label-dy, label-orient, legend, name, offset, opacity, repeat, shape, show-connection, width
custom	bandfill, channels, connection-label, depth, fill, height, image, input-style, label, label-anchor, label-angle, label-dx, label-dy, label-orient, legend, name, offset, opacity, repeat, shape, show-connection, show-relu, width, widths, xlabel, ylabel, zlabel
deconv	channels, connection-label, depth, fill, height, image, label, label-anchor, label-angle, label-dx, label-dy, label-orient, legend, name, offset, opacity, repeat, shape, show-connection, width
fc	channels, connection-label, depth, fill, height, image, label, label-anchor, label-angle, label-dx, label-dy, label-orient, legend, name, offset, opacity, repeat, shape, show-connection
gap	channels, connection-label, depth, fill, height, image, label, label-anchor, label-angle, label-dx, label-dy, label-orient, legend, name, offset, opacity, repeat, shape, show-connection
input	channels, connection-label, depth, fill, height, image, input-style, label, label-anchor, label-angle, label-dx, label-dy, label-orient, legend, name, offset, opacity, repeat, shape, show-connection, width
output	channels, classes, connection-label, depth, fill, height, image, label, label-anchor, label-angle, label-dx, label-dy, label-orient, legend, name, offset, opacity, repeat, shape, show-connection, width
pool	channels, connection-label, depth, fill, height, image, label, label-anchor, label-angle, label-dx, label-dy, label-orient, legend, name, offset, opacity, repeat, shape, show-connection
softmax	channels, classes, connection-label, depth, fill, height, image, label, label-anchor, label-angle, label-dx, label-dy, label-orient, legend, name, offset, opacity, repeat, shape, show-connection, width
sum	channels, connection-label, depth, fill, height, label, label-anchor, label-angle, label-dx, label-dy, label-orient, legend, name, offset, opacity, radius, show-connection, stroke, symbol
unpool	channels, connection-label, depth, fill, height, image, label, label-anchor, label-angle, label-dx, label-dy, label-orient, legend, name, offset, opacity, repeat, shape, show-connection

14.2 Connection options

arrive-offset · clearance · color · dash · from · label · layers · legend · mode · opacity · pos · thickness · to · touch-layer · type

14.3 Group options

color · from · label · offset · to

14.4 Arguments to draw-network

```
#draw-network(  
  layers,  
  connections: (),  
  groups: (),  
  palette: "warm",  
  show-legend: false,  
  legend-title: "Layers",  
  main-legend: none,  
  scale: 100%,  
  stroke-thickness: 1,  
  depth-multiplier: 0.3,  
  lane-unit: 0.75,  
  shape-scale: (spatial: (1.2, -3.2), channels: (0.075, 0.0)),  
  auto-gap: 0.55,  
  show-relu: false,  
)
```

14.5 Coverage

Every option exported by the package is described somewhere above.

Where to go next

Complete architectures are in `examples/` in the repository, and their rendered output in `gallery/`. They are ordinary Typst files, meant to be read and copied:

examples/networks/ AlexNet, LeNet-5, VGG16, VGG19, ResNet18, U-Net, FCN-8 and YOLO26-n, plus five RGB-IR fusion variants of the last one and a figure opening up the fusion operators themselves.

examples/features/ the figures from this manual's ancestors, including every predefined layer type side by side in both palettes.

examples/imported/ figures generated from a traced PyTorch model rather than written by hand, using `tools/import_model.py`.

The YOLO26-n example is the one to read if you want to see the automatic machinery carrying a whole figure: every block states its tensor shape, every gap is automatic, every lane height is automatic, and the three detection heads are a depth-mode branch. Nothing in it is sized or positioned by hand.